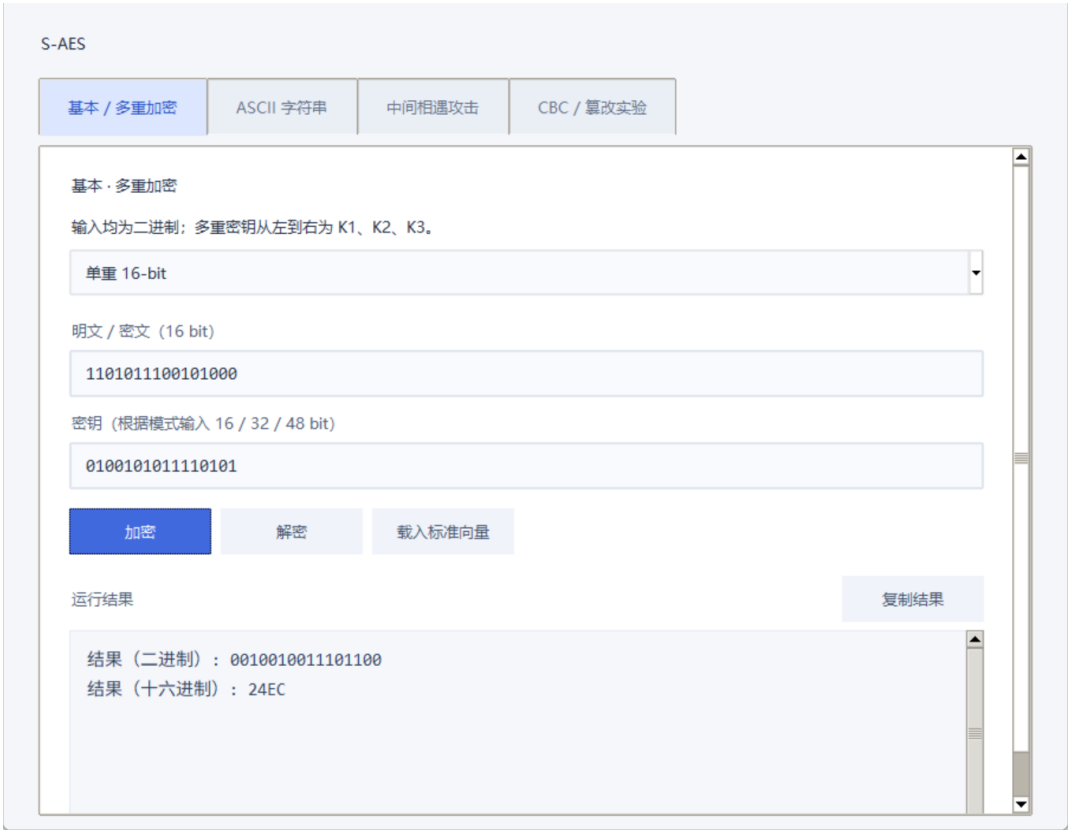
作业二——测试结果

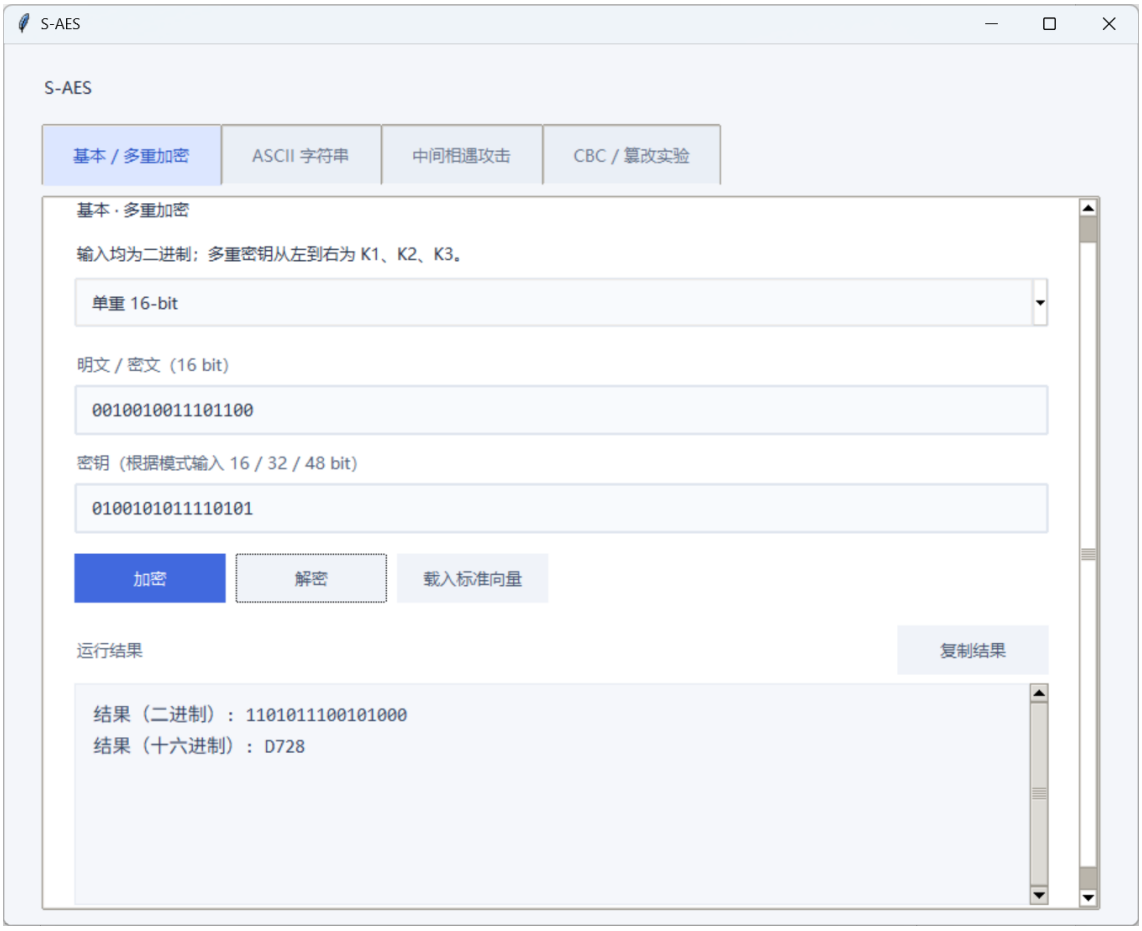
第 1 关：基本测试

在程序“基本/多重加密”页面选择单重加密模式，输入 16 bit 明文与 16 bit 密钥，进行加密和解密测试。测试数据及结果如下。

项目	值
明文 P	1101011100101000
密钥 K	0100101011110101
初始轮密钥 K_0	4AF5
第一轮密钥 K_1	DD28
第二轮密钥 K_2	87AF
加密结果 C	0010010011101100
密文十六进制	24EC
解密结果 $D(K,C)$	1101011100101000

输入明文与密钥后，程序输出 16 bit 密文。随后将该密文作为解密输入，使用相同密钥进行解密，恢复结果与原始明文一致。加密和解密的具体截图分别如下：





第 2 关：交叉测试

1. 单重加密（16-bit）交叉测试

交叉测试要求双方采用相同的 S-AES 算法流程、替换盒、列混淆矩阵及状态排列方式。单重加密采用的主要约定如下：

项目	本项目约定
S 盒	9 4 A B D 1 8 5 6 2 0 3 C E F 7
列混淆矩阵	[[1,4],[4,1]]
逆列混淆矩阵	[[9,2],[2,9]]
有限域模多项式	$x^4 + x + 1$
状态排列	四个半字节按列排列
数据及密钥长度	均为 16 bit

选取相同的明文与密钥后，我组加密输出的密文如下：



对方加密输出的密文如下：



可见明文和密钥相同的情况下，可输出相同的密文。

2. 双重加密交叉测试

选取相同的明文与密钥后，我组加密输出的密文如下：



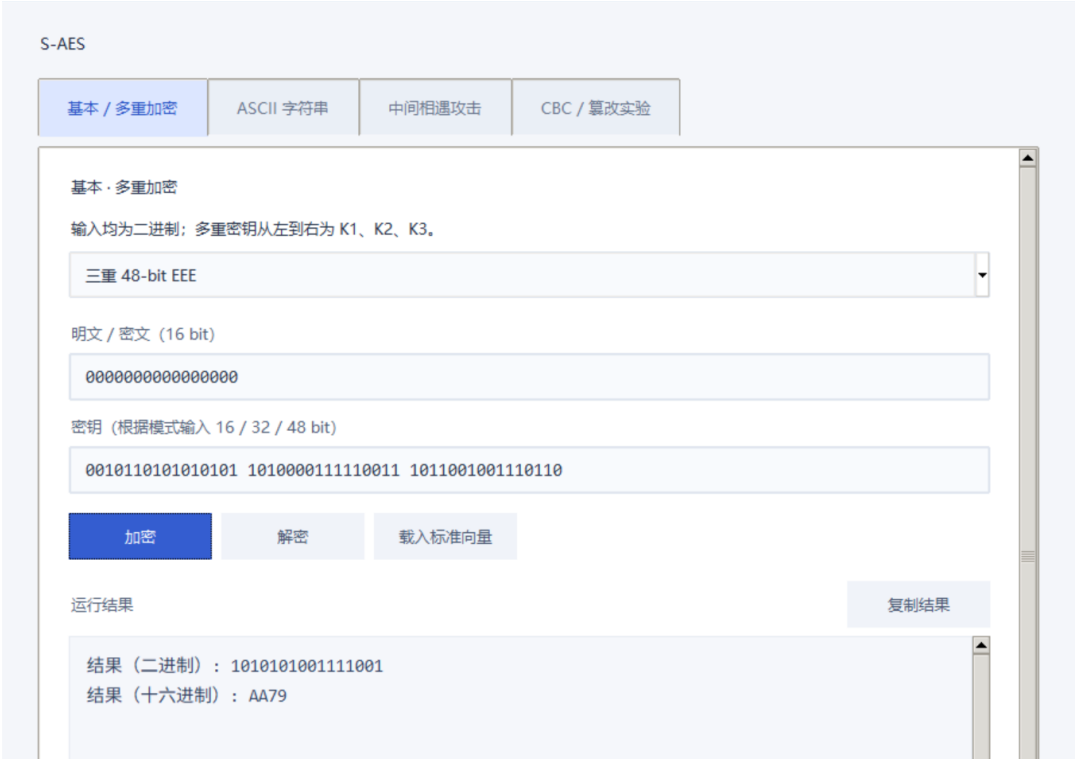
对方加密输出的密文如下：



可见明文和密钥相同的情况下，可输出相同的密文。

3. 三重加密交叉测试

选取相同的明文与密钥后，我组加密输出的密文如下：



对方加密输出的密文如下：



可见明文和密钥相同的情况下，可输出相同的密文。

第 3 关：扩展功能

在“ASCII 字符串”页面输入 ASCII 明文，程序将其转换为字节数据，每 2 Bytes 组成一个 16 bit 分组后进行加密。

测试使用的明文和密钥为：

明文：Hello S-AES!

密钥：0100101011110101

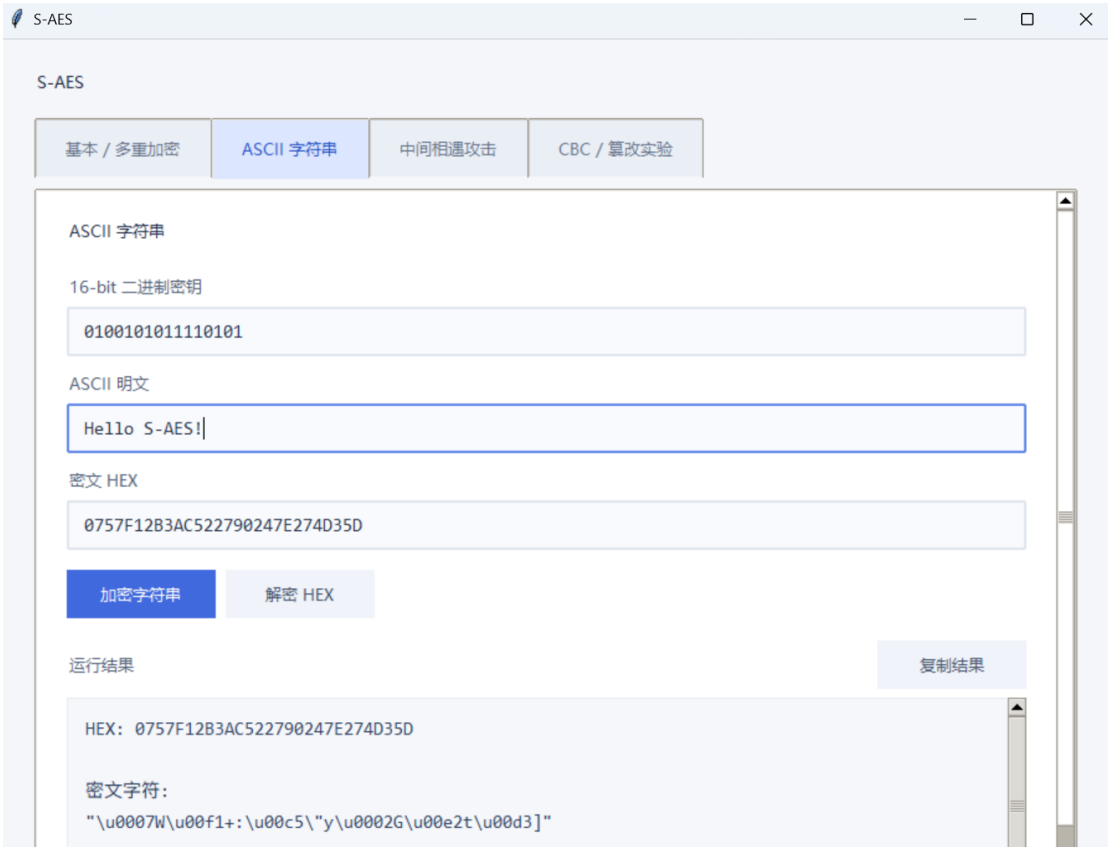
由此得到的完整密文为：

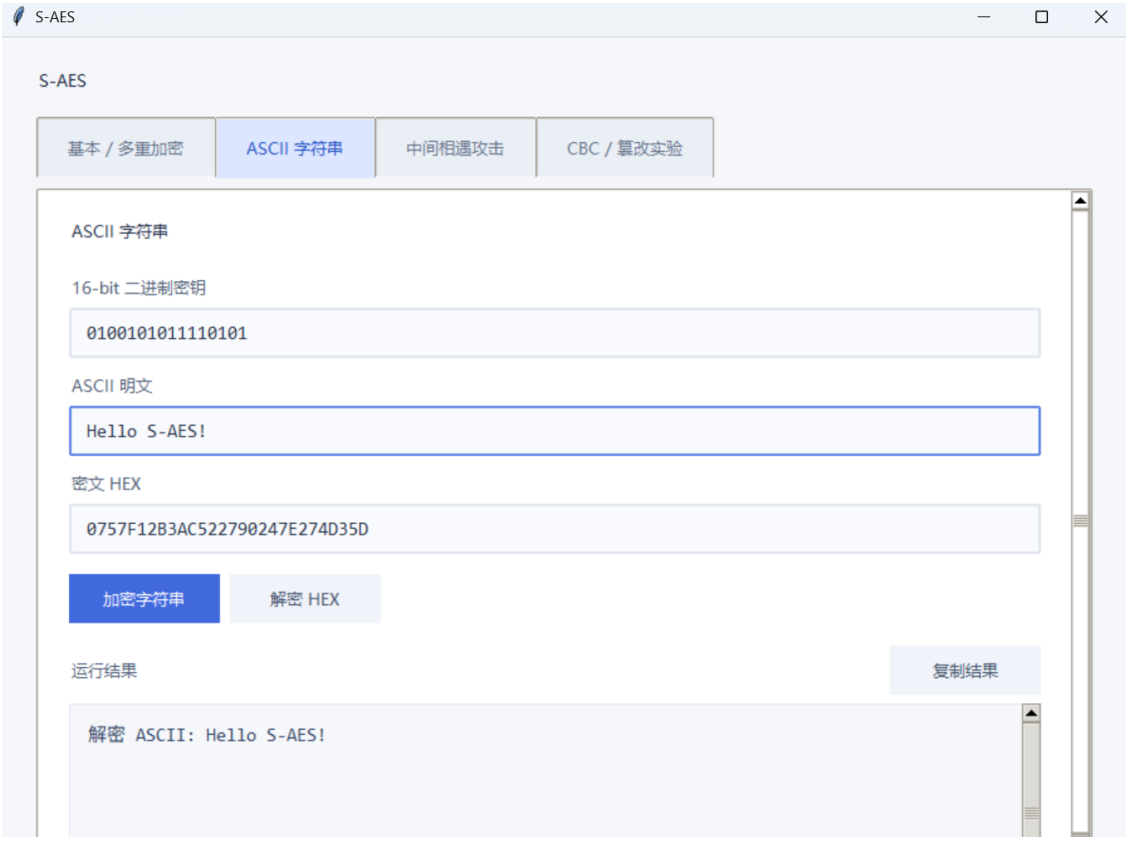
0757F12B3AC522790247E274D35D

将上述密文与相同密钥输入解密功能，恢复结果为：

Hello S-AES!

加密和解密的具体截图分别如下：





第 4 关：多重加密

1. 双重加密

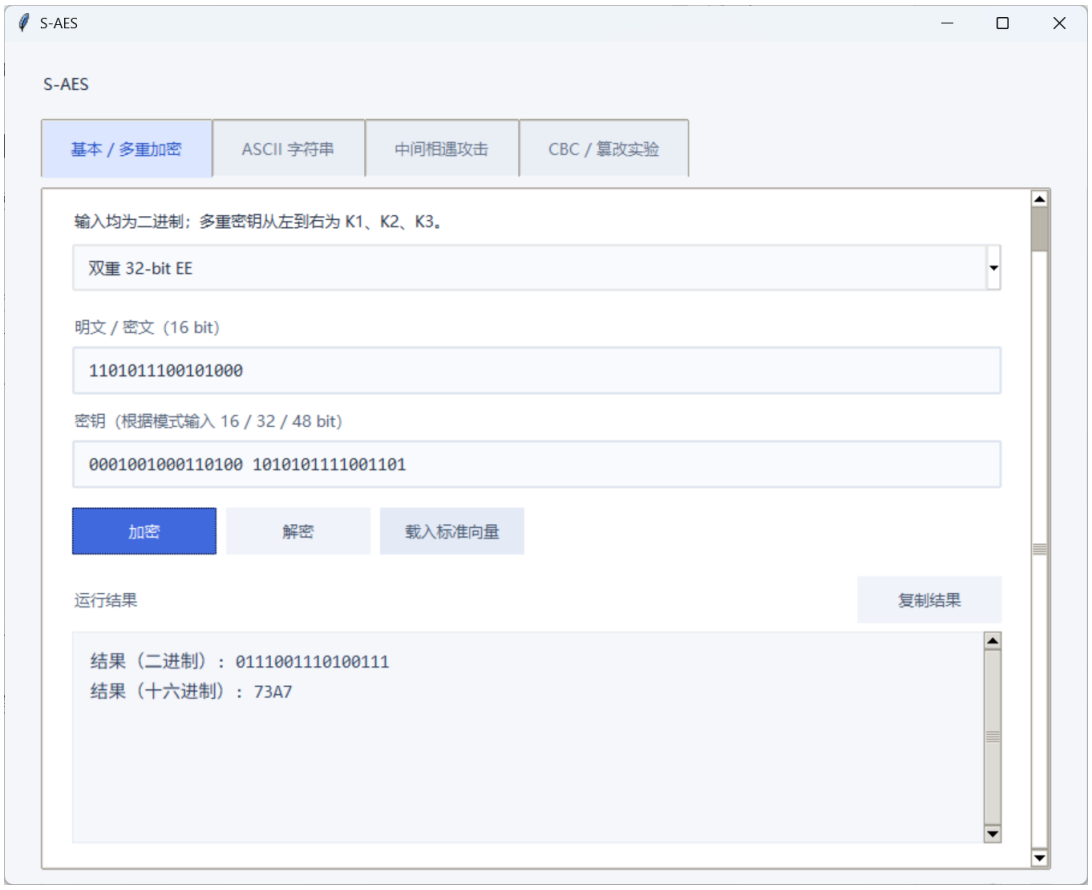
双重加密使用 32 bit 组合密钥，由两个 16 bit 子密钥K_1、K_2组成，加密顺序为：

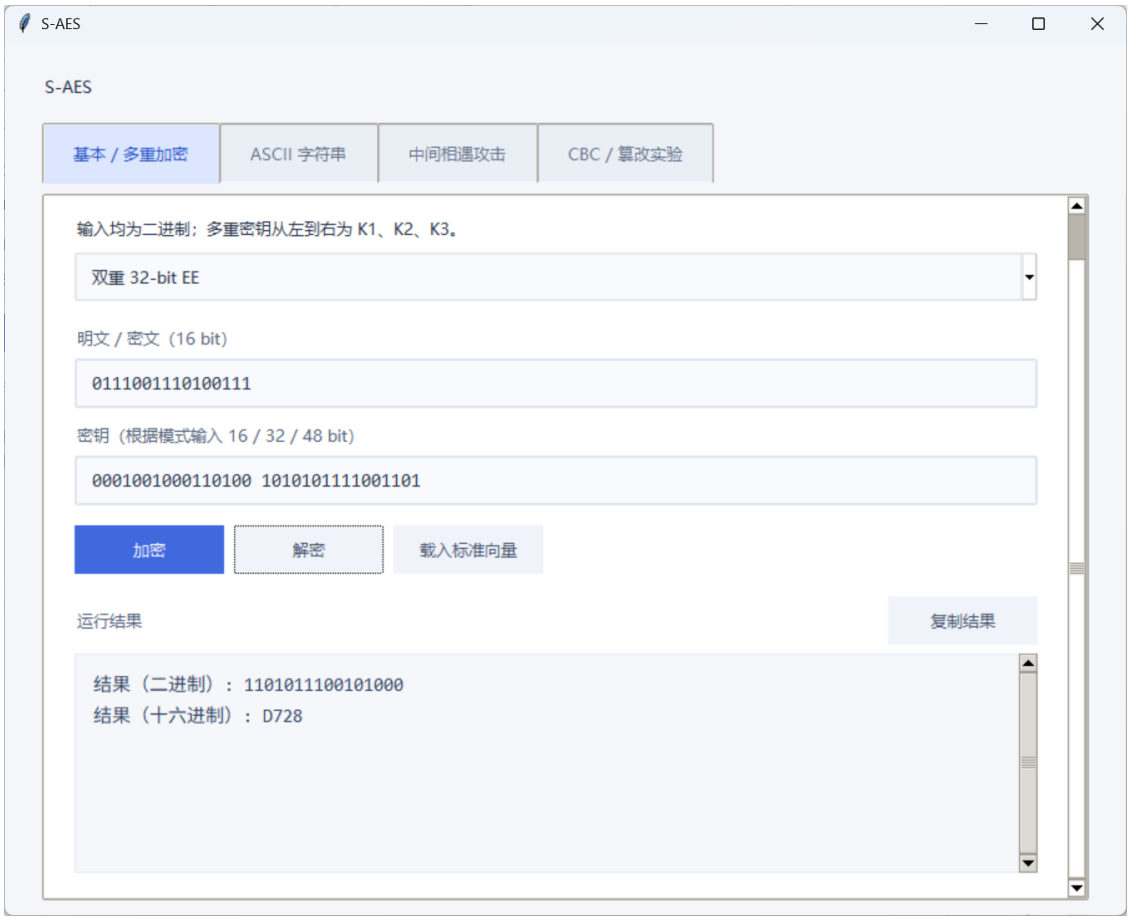
$$C = E(K_2, E(K_1, P))$$

解密时按照相反顺序依次使用K_2、K_1。具体采用的明文、密钥和输出结果如下：

项目	值
明文 P	1101011100101000
子密钥 K_1	0001001000110100
子密钥 K_2	1010101111001101
最终密文（二进制）	0111001110100111
双重解密结果	1101011100101000

加密和解密结果的截图分别如下：





2. 中间相遇攻击

使用组合密钥 1234ABCD 构造以下三组已知明密文对：

编号	明文 P	密文 C
1	1234	A36B
2	5678	9408
3	9ABC	CD4D

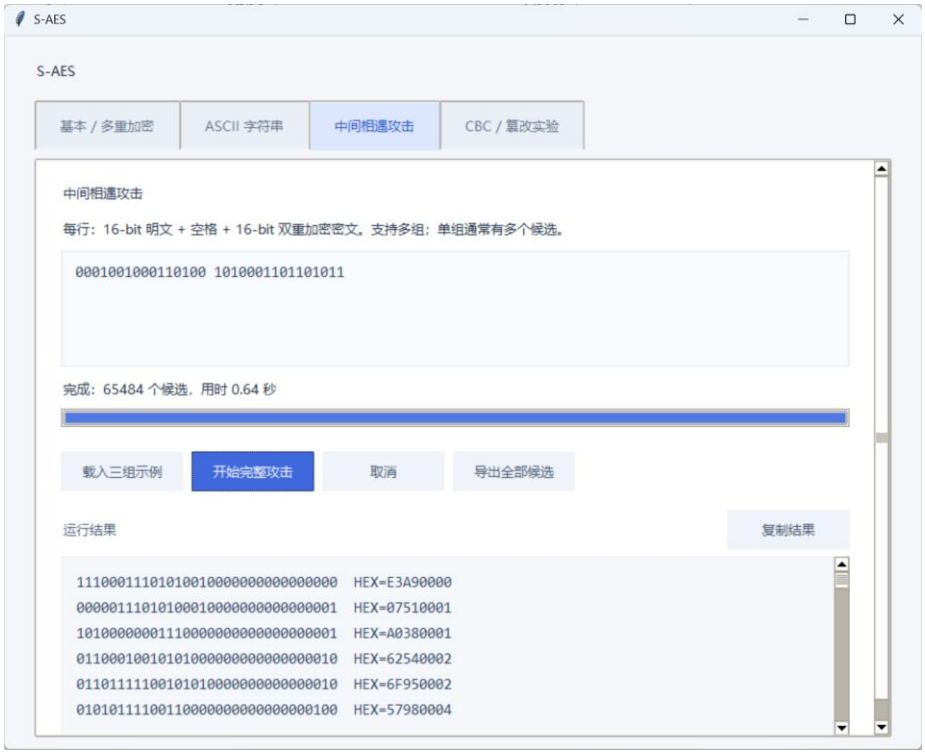
程序枚举全部 65,536 个K_1，保存中间值 $E(K_1, P)$；再枚举全部 65,536 个K_2，计算 $D(K_2, C)$ 并匹配中间值，最后利用其余明密文对过滤候选密钥。接下来分别使用一组、两组和三组明密文对，得到以下结果。

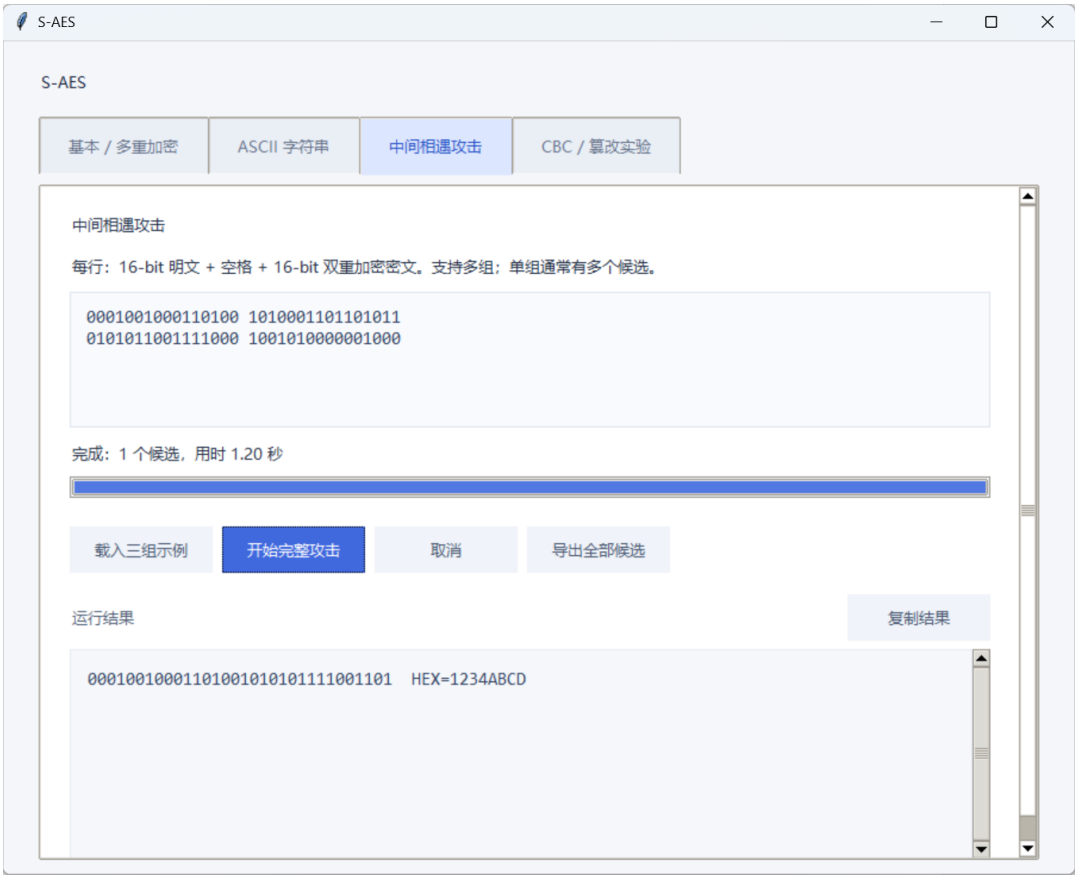
已知明密文对数量	候选密钥数量	是否包含原始密钥	本次运行耗时
1 组	65,484	是	0.64s
2 组	1	是	1.20s
3 组	1	是	1.22s

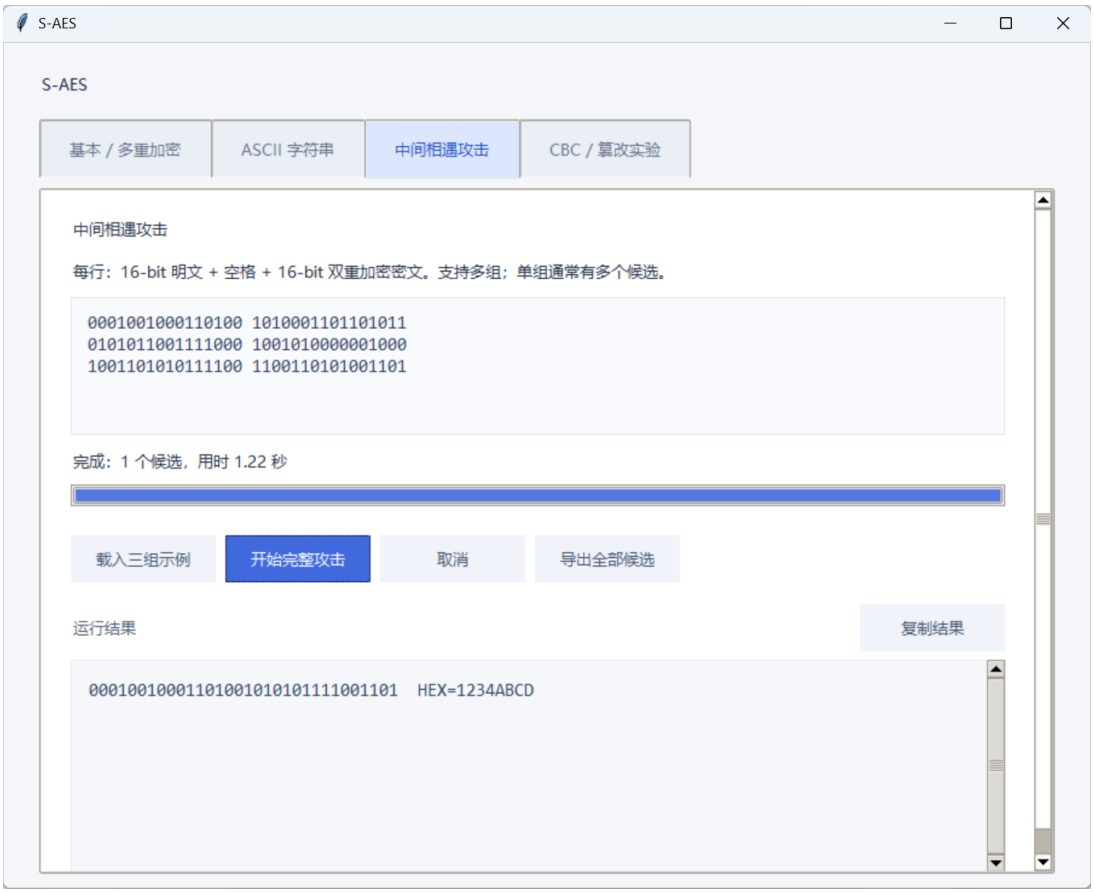
其中使用 2 组、3 组观测得到的唯一候选密钥为：

二进制：00010010001101001010101111001101；十六进制：1234ABCD

该结果与实验设置的原始密钥一致。而单组明密文对产生了大量候选密钥，通过增加观测后，能够唯一恢复密钥。具体截图如下：




S-AES
S-AES
基本 / 多重加密
ASCII 字符串
中间相遇攻击
CBC / 篡改实验
中间相遇攻击
每行：16-bit 明文 + 空格 + 16-bit 双重加密密文。支持多组；单组通常有多个候选。
0001001000110100 1010001101101011
0101011001111000 1001010000001000
完成：1 个候选，用时 1.20 秒
载入三组示例
开始完整攻击
取消
导出全部候选
运行结果
复制结果
00010010001101001010101111001101 HEX=1234ABCD


S-AES
S-AES
基本 / 多重加密
ASCII 字符串
中间相遇攻击
CBC / 篡改实验
中间相遇攻击
每行：16-bit 明文 + 空格 + 16-bit 双重加密密文。支持多组；单组通常有多个候选。
0001001000110100 1010001101101011
0101011001111000 1001010000001000
1001101010111100 1100110101001101
完成：1 个候选，用时 1.22 秒
载入三组示例
开始完整攻击
取消
导出全部候选
运行结果
复制结果
00010010001101001010101111001101 HEX=1234ABCD

3. 三重加密

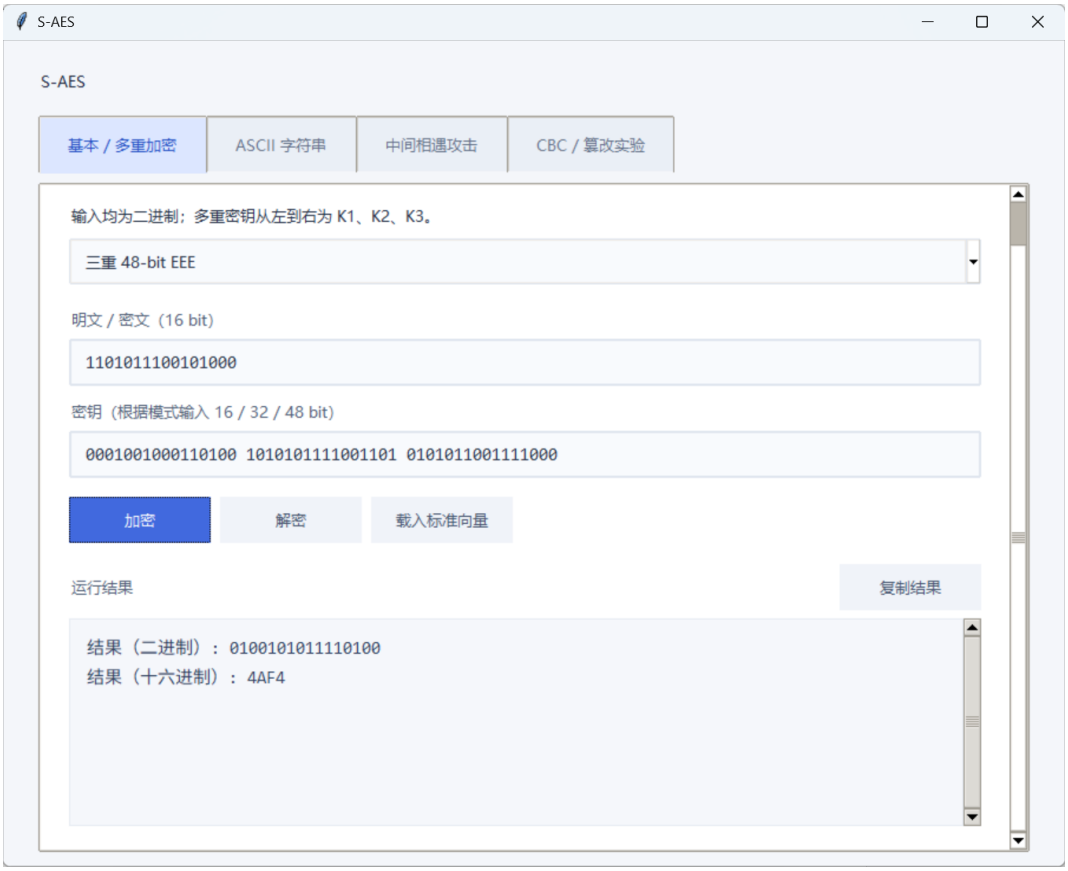
本项目选择 48 bit 密钥模式，由三个 16 bit 子密钥K_1、K_2、K_3组成，采用 EEE 方式进行三重加密：

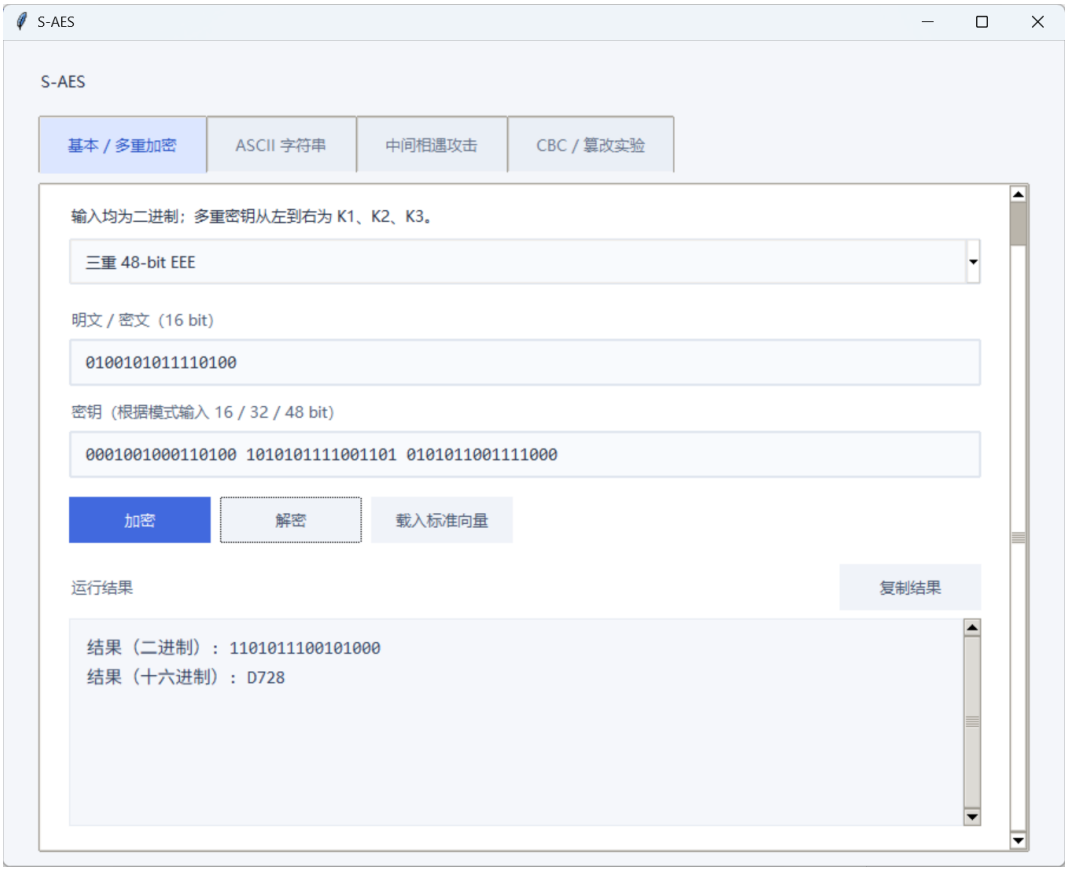
$$C = E(K_3, E(K_2, E(K_1, P)))$$

解密时按照相反顺序依次使用K_3、K_2、K_1。具体采用的明文、密钥和输出结果如下：

项目	值
明文 P	1101011100101000
子密钥 K_1	0001001000110100
子密钥 K_2	1010101111001101
子密钥 K_3	0101011001111000
48 bit 组合密钥	1234ABCD5678
最终密文（二进制）	0100101011110100
三重解密结果	1101011100101000

加密和解密结果的截图分别如下：





第 5 关：工作模式

1. CBC 模式加解密

在“CBC / 篡改实验”页面，使用 CBC 模式对较长 ASCII 消息进行加密。程序支持生成 16 bit 随机初始向量 IV，但本次测试选取 IV 为 1234。加密方与解密方使用相同密钥和 IV，在这里不将 IV 自动拼接到密文中，因此双方需要在传输密文时另外共享 IV。其他参数与加解密结果如下：

项目	值
明文	Hello CBC mode!
密钥	4AF5
密钥（二进制）	0100101011110101
IV	1234
原始明文长度	15 Bytes
填充后长度	16 Bytes，共 8 个分组
密文 HEX	E3344527B8FBB961F913BB45BAC29FA9
解密结果	Hello CBC mode!

S-AES

基本 / 多重加密 | ASCII 字符串 | 中间相遇攻击 | CBC / 篡改实验

16-bit 二进制密钥

0100101011110101

共享 IV（16 bit；加密前可生成随机 IV）

0001001000110100

ASCII 明文

Hello CBC mode!

原始密文 HEX（可复制并分享，IV 需单独共享）

E3344527B8FBB961F913BB45BAC29FA9

篡改后密文 HEX（可直接编辑，实现替换或修改）

E3344527B8FBB961F913BB45BAC29FA9

随机 IV | CBC 加密 | CBC 解密 | 翻转 C1 一位 | 对比篡改结果

运行结果 复制结果

IV: 0001001000110100
密文: E3344527B8FBB961F913BB45BAC29FA9

基本 / 多重加密 | ASCII 字符串 | 中间相遇攻击 | CBC / 篡改实验

CBC · 篡改实验

16-bit 二进制密钥

0100101011110101

共享 IV（16 bit；加密前可生成随机 IV）

0001001000110100

ASCII 明文

Hello CBC mode!

原始密文 HEX（可复制并分享，IV 需单独共享）

E3344527B8FBB961F913BB45BAC29FA9

篡改后密文 HEX（可直接编辑，实现替换或修改）

E3344527B8FBB961F913BB45BAC29FA9

随机 IV | CBC 加密 | CBC 解密 | 翻转 C1 一位 | 对比篡改结果

运行结果 复制结果

解密 HEX: 48656C6C6F20434243206D6F646521
解密字节: b'Hello CBC mode!'
文本（不可解码字节显示为）: Hello CBC mode!

保持明文与密钥不变，将加密 IV 改为 1235，得到的密文为：

6338352948F5796709162B48FACDDFAA



可见在改变了 IV 后，相同明文产生了不同密文。

2. 修改密文分组后的解密结果

对原密文的第一个字节翻转一位，使首个密文分组由 E334 改为 E234，其余分组保持不变。对其进行解密后的结果如图所示：



篡改前和篡改后的解密效果对比如下：

明文分组	修改前解密结果	修改后解密结果	对比结果
P_1	4865	4A05	当前分组被扰乱
P_2	6C6C	6D6C	对应位发生翻转
P_3	6F20	6F20	相同
P_4	4342	4342	相同
P_5	4320	4320	相同
P_6	6D6F	6D6F	相同
P_7	6465	6465	相同
P_8	2101	2101	相同，包含填充字节

首个密文分组的修改差值为 0100，第二个明文分组的变化同样为：

$$6C6C\ XOR\ 6D6C\ =\ 0100$$

因此，修改C_1后，P_1被扰乱，P_2的对应位翻转，P_3及后续分组保持不变。

3. 替换密文分组后的解密结果

将原密文的第一个分组C_1替换为第二个分组C_2，即将 E334 替换为 4527，其余分组不变。对其进行解密后的结果如图所示：



篡改前和篡改后的解密效果对比如下：

明文分组	替换前	替换后	对比结果
P_1	4865	9D6C	当前分组被扰乱
P_2	6C6C	CA7F	随前一密文分组变化
$P_3 \sim P_8$	6F20 4342 4320 6D6F 6465 2101	6F20 4342 4320 6D6F 6465 2101	全部相同



第二个明文分组的异或差值为：

$$6C6C\ XOR\ CA7F\ =\ A613$$

原C_1与替换后C_1的异或差值为：

$$E334\ XOR\ 4527\ =\ A613$$

两者一致，符合 CBC 解密中前一密文分组参与异或的计算关系。规律：修改 C_i 使P_i扰乱，P_{i+1}在相应位翻转，后续分组恢复。

两种篡改实验均表明，修改某个密文分组会影响当前及下一个明文分组，之后的分组恢复正常。CBC 能够对消息进行分组链接加密，但本身不提供完整性校验，无法仅凭解密是否成功判断密文是否遭到篡改。